

Chapter 17: Surrogate Models

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

Lecture Notes

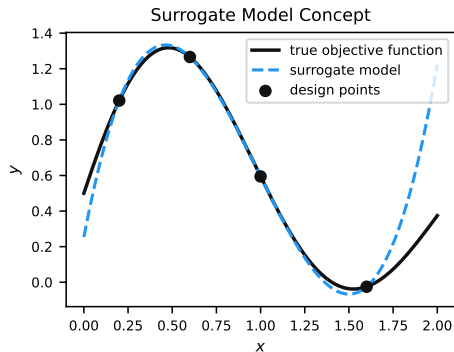
June 12, 2026

Outline

- 1 Introduction: Fitting Surrogate Models
- 2 Linear Models
- 3 Basis Functions
 - Polynomial Basis Functions
 - Sinusoidal Basis Functions
 - Radial Basis Functions
- 4 Fitting Noisy Objective Functions
- 5 Model Selection
 - Holdout Method
 - Cross-Validation
 - Bootstrap
 - AIC and BIC
- 6 Multifidelity Surrogate Models
- 7 Summary & Exercises

Why Surrogate Models?

- Many real-world objective functions are **expensive** to evaluate (e.g. CFD simulations, physical experiments).
- A **surrogate model** $\hat{f}_\theta$ is a *smooth, cheap* approximation fitted to a finite set of evaluations.
- The surrogate can then be *optimized in place of* the true function.
- Key properties required:
 - Inexpensive to evaluate.
 - Smooth enough to apply gradient-based or Bayesian methods.
 - Generalises well away from observed design points.



Surrogate (blue dashed) fitted to design points (dots);
deviates from truth far from data.

Problem Setup

Design Points and Evaluations

Suppose we have m design points

$$\mathbf{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}\} \quad (17.1)$$

and associated objective function evaluations

$$\mathbf{y} = \{y^{(1)}, y^{(2)}, \dots, y^{(m)}\} \quad (17.2)$$

Surrogate Predictions

For parameter vector θ , the model predicts

$$\hat{\mathbf{y}} = \{\hat{f}_{\theta}(\mathbf{x}^{(1)}), \hat{f}_{\theta}(\mathbf{x}^{(2)}), \dots, \hat{f}_{\theta}(\mathbf{x}^{(m)})\} \quad (17.3)$$

Fitting Objective (Regression)

$$\underset{\theta}{\text{minimize}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_p \quad (17.4)$$

Linear Model

Definition (Eq. 17.5)

$$\hat{f} = w_0 + \mathbf{w}^\top \mathbf{x}, \quad \boldsymbol{\theta} = \{w_0, \mathbf{w}\}$$

- $w_0 \in \mathbb{R}$: bias (intercept).
- $\mathbf{w} \in \mathbb{R}^n$: weight vector.
- For an n -dimensional design space: $n + 1$ parameters, requires at least $n + 1$ samples.

Compact Form (Eq. 17.6)

Prepend 1 to $\mathbf{x}$ and absorb w_0 into $\boldsymbol{\theta} = [w_0, \mathbf{w}]$:

$$\hat{f} = \boldsymbol{\theta}^\top \mathbf{x}$$

Design Matrix (Eq. 17.9)

$$\begin{bmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(n+1)})^\top \end{bmatrix}$$

Linear Regression: Normal Equations

Optimization Problem (Eq. 17.7–17.8)

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 \equiv \underset{\boldsymbol{\theta}}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

Closed-Form Solution (Eq. 17.10)

Linear regression has an **analytic solution** via the Moore–Penrose pseudoinverse:

$$\boldsymbol{\theta} = \mathbf{X}^+ \mathbf{y}$$

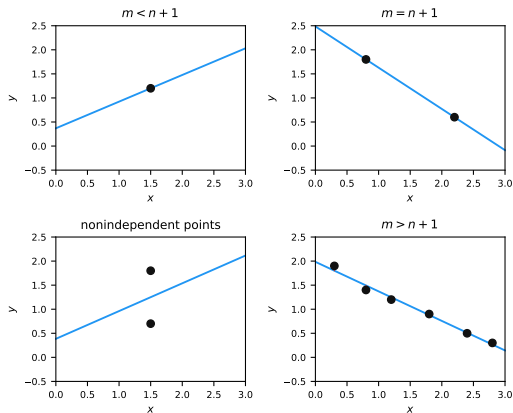
where $\mathbf{X}^+ = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ when $\mathbf{X}$ has full column rank.

Normal Equation (Exercise 17.1)

Setting $\nabla \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 = \mathbf{0}$ yields:

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$$

Linear Regression: Cases



Four cases: $m < n + 1$ (underdetermined), $m = n + 1$ (exact), non-independent, $m > n + 1$ (overdetermined — vertical bars show residuals).

- $m < n + 1$: underdetermined; infinitely many solutions. Pseudoinverse gives minimum-norm solution.
- $m = n + 1$: unique solution (if $\mathbf{X}$ invertible).
- Non-independent: $\mathbf{X}$ is singular; pseudoinverse still gives a unique solution.
- $m > n + 1$: overdetermined (common case); pseudoinverse minimises sum of squared residuals.

Key Property

The pseudoinverse always returns a *unique* solution for any non-empty point configuration.

Linear Regression in Python

```
1 import numpy as np
2
3 def design_matrix(X):
4     """X: list of 1-D design points.
5     Returns matrix with leading column of ones."""
6     m = len(X)
7     return np.column_stack([np.ones(m), X])
8
9 def linear_regression(X, y):
10    """Fit linear model:  $\theta = X^+ y$ .
11    Returns  $\theta = [w_0, w_1, \dots, w_n]$ ."""
12    Xmat = design_matrix(X)
13    # np.linalg.lstsq uses the pseudoinverse
14    theta, _, _, _ = np.linalg.lstsq(Xmat, y, rcond=
        None)
15    return theta
16
17 # Example: 5 points in 1-D
18 X = np.array([0.1, 0.4, 0.7, 1.2, 1.8])
19 y = np.array([0.3, 0.8, 1.2, 1.9, 2.5])
20 theta = linear_regression(X, y)
21 print(f"theta = {theta}")
22 # Prediction at new point
23 x_new = 1.0
24 y_pred = theta[0] + theta[1] * x_new
```

- `np.linalg.lstsq` computes the minimum-norm least-squares solution.

- Equivalent to $\theta = X^+ y$.

- **Numerical example:**

$$X = [1, 2, 3, 4], \quad y = [0, 5, 4, 6]$$

Fit polynomial degree $k = 1$:

$$\theta = \begin{bmatrix} -0.5 \\ 1.7 \end{bmatrix}$$

$$\text{so } \hat{f}(x) = -0.5 + 1.7x.$$

From Linear to Basis-Function Models

Linear Model as Basis Functions (Eq. 17.11)

$$\hat{f}(\mathbf{x}) = \theta_1 x_1 + \cdots + \theta_n x_n = \sum_{i=1}^n \theta_i x_i = \boldsymbol{\theta}^\top \mathbf{x}$$

Linear model: basis functions $b_i(\mathbf{x}) = x_i$.

General Basis-Function Model (Eq. 17.12)

$$\hat{f}(\mathbf{x}) = \theta_1 b_1(\mathbf{x}) + \cdots + \theta_q b_q(\mathbf{x}) = \sum_{i=1}^q \theta_i b_i(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{b}(\mathbf{x})$$

where $b_1, \dots, b_q$ are arbitrary *basis functions*.

Regression with Basis Functions (Eq. 17.13–17.15)

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \|\mathbf{y} - \mathbf{B}\boldsymbol{\theta}\|_2^2, \quad \mathbf{B} = \begin{bmatrix} \mathbf{b}(\mathbf{x}^{(1)})^\top \\ \mathbf{b}(\mathbf{x}^{(2)})^\top \\ \vdots \end{bmatrix}, \quad \boldsymbol{\theta} = \mathbf{B}^+ \mathbf{y}$$

Basis-Function Regression in Python

```
1 import numpy as np
2
3 def regression(X, y, bases, lam=0.0):
4     """
5     Fit surrogate model using basis functions.
6     X      : array of m design points
7     y      : array of m objective values
8     bases  : list of q callable basis functions b_i(x)
9     lam    : L2 regularisation parameter (default 0)
10    Returns: callable model(x)
11    """
12    B = np.array([[b(x) for b in bases] for x in X]) # m x q
13    if lam == 0.0:
14        theta, _, _, _ = np.linalg.lstsq(B, y, rcond=None)
15    else:
16        # theta = (B'B + lam*I)^{-1} B'y
17        A = B.T @ B + lam * np.eye(len(bases))
18        theta = np.linalg.solve(A, B.T @ y)
19    return lambda x: sum(theta[i] * bases[i](x) for i in range(len(theta)))
```

Listing 2: General basis-function regression (Algorithm 17.2 equivalent)

- The basis matrix $\mathbf{B} \in \mathbb{R}^{m \times q}$ replaces the design matrix $\mathbf{X}$.
- Same pseudoinverse solve; $O(mq^2)$ complexity.
- `lam` adds L_2 regularisation (see Section 17.4).

Polynomial Basis Functions

1-D Polynomial Model of Degree k (Eq. 17.16)

$$\hat{f}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_k x^k = \sum_{i=0}^k \theta_i x^i$$

Basis functions: $b_i(x) = x^i$ for $i = 0, 1, \dots, k$.

2-D Polynomial of Degree k (Eq. 17.17)

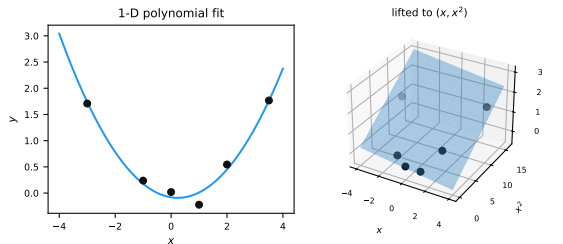
$$b_{ij}(\mathbf{x}) = x_1^i x_2^j, \quad i, j \in \{0, \dots, k\}, \quad i + j \leq k$$

- Number of basis functions: $\binom{n+k}{k}$ for n -dimensional input.
- Total parameters for $n = 2, k = 2$: 6 (constant, x_1 , x_2 , x_1^2 , $x_1 x_2$, x_2^2).

Key Insight

A polynomial model is *linear in the higher-dimensional feature space*. Fitting reduces to standard linear regression on features $\{x^i\}$.

Polynomial Regression: Higher-Dimensional View



Left: polynomial fit in 1-D. Right: the same fit is a hyperplane in the lifted space (x, x^2) .

- A degree- k polynomial in $\mathbb{R}^n$ is **linear** in $\mathbb{R}^{k+1}$.
- Taylor's theorem: any smooth function can be *closely approximated* by a polynomial of sufficient degree.
- Pitfall: high-degree polynomials can **overfit** in sparse regions.

```
1 # 1-D polynomial bases up to degree k
2 def poly_bases_1d(k):
3     return [lambda x, p=p: x**p
4             for p in range(k+1)]
5
6 # Example: degree-2 fit
7 bases = poly_bases_1d(2)
8 X = np.array([-2, -1, 0, 1, 2])
9 y = np.array([4.1, 1.0, 0.1, 0.9, 4.0])
10 model = regression(X, y, bases)
```

Sinusoidal / Fourier Basis Functions

Fourier Series (Eq. 17.18–17.20)

Any integrable univariate function f on interval $[a, b]$ can be represented as:

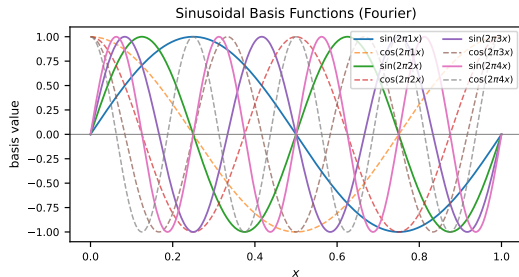
$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} \left[a_k \cos(2\pi kx) + b_k \sin(2\pi kx) \right]$$

where

$$a_k = \frac{2}{b-a} \int_a^b f(x) \cos(2\pi kx) dx$$

$$b_k = \frac{2}{b-a} \int_a^b f(x) \sin(2\pi kx) dx$$

Truncating to K terms gives a surrogate with $2K + 1$ parameters.



Sinusoidal basis functions for $k = 1, 2, 3, 4$.

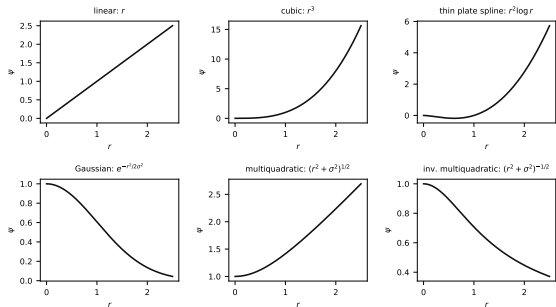
- Useful when the objective has *periodic* or oscillatory structure.
- Can combine with polynomial bases.
- Exact representation for periodic functions (complete basis over L^2).

Radial Basis Functions (RBF)

Definition

A **radial basis function** (RBF) depends only on the distance $r = \|\mathbf{x} - \mathbf{c}\|$ from a centre $\mathbf{c}$:

$$b(\mathbf{x}) = \psi(\|\mathbf{x} - \mathbf{c}\|)$$



Name	$\psi(r)$
Linear	r
Cubic	r^3
Thin plate spline	$r^2 \log r$
Gaussian	$e^{-r^2/2\sigma^2}$
Multiquadric	$(r^2 + \sigma^2)^{1/2}$
Inv. multiquadric	$(r^2 + \sigma^2)^{-1/2}$

Common RBF kernels as a function of radius r .

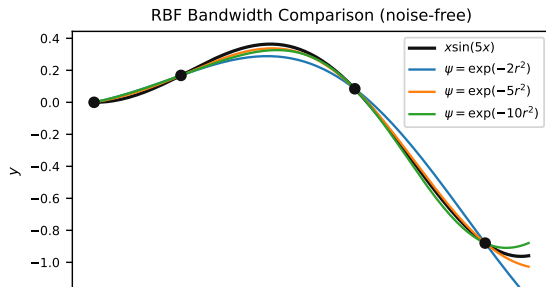
RBF Surrogate Construction

RBF Model with m Design Points as Centres

Place one RBF at each design point $\mathbf{x}^{(i)}$:

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^m \theta_i \psi(\|\mathbf{x} - \mathbf{x}^{(i)}\|)$$

The $m \times m$ basis matrix $\mathbf{B}$ has entries $B_{ij} = \psi(\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|)$.



```
1 # Gaussian RBF bases centred at design points
2 def make_rbf_bases(X_train, c=5.0):
3     return [lambda x, xi=xi:
4             np.exp(-c*(x-xi)**2)
5             for xi in X_train]
6
7 # Fit
8 bases = make_rbf_bases(X_train)
9 model = regression(X_train, y_train,
10                   bases, lam=0.0)
```

- Bandwidth c controls locality.
- Large c : narrow, interpolates tightly.

Noisy Observations and L_2 Regularisation

Problem

When objective evaluations are **noisy**: $y^{(i)} = f(\mathbf{x}^{(i)}) + \epsilon^{(i)}$, an unregularised surrogate may **overfit** the noise.

Regularised Regression (L_2 / Ridge, Eq. 17.24)

Add a penalty on parameter magnitude:

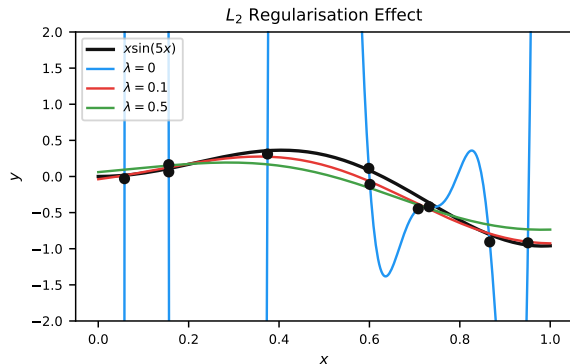
$$\underset{\boldsymbol{\theta}}{\text{minimize}} \quad \|\mathbf{y} - \mathbf{B}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

- $\lambda \geq 0$: *smoothing parameter* (hyperparameter).
- $\lambda = 0$: no regularisation (standard regression).

Closed-Form Solution (Eq. 17.25)

$$\boldsymbol{\theta} = (\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{I})^{-1} \mathbf{B}^\top \mathbf{y}$$

Effect of Regularisation



10 noisy samples of $x\sin(5x)$. Large λ over-smooths; $\lambda = 0$ overfits.

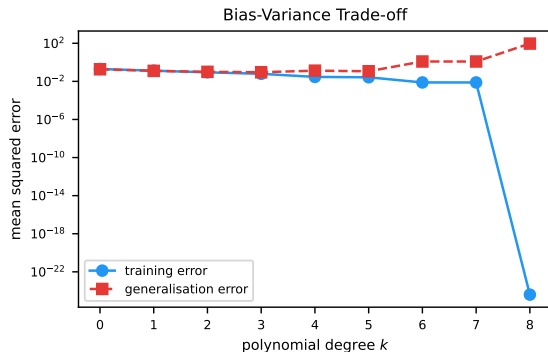
Cross-validation finds optimal $\lambda \approx 0.1$.

```
1 import numpy as np
2
3 def regression_regularised(X, y, bases, lam):
4     """Ridge regression with L2 penalty lam."""
5     B = np.array([[b(x) for b in bases]
6                   for x in X]) # m x q
7     A = B.T @ B + lam * np.eye(len(bases))
8     theta = np.linalg.solve(A, B.T @ y)
9     return lambda x: sum(theta[i]*bases[i](x)
10                          for i in range(len(theta)))
11
12 # Fit with lambda=0.1
13 X = np.linspace(0, 1, 10)
14 y = X*np.sin(5*X) + 0.1*np.random.randn(10)
15 bases = make_rbf_bases(X, c=5.0)
16 model = regression_regularised(X, y, bases,
17                               lam=0.1)
```

Listing 3: Regularised RBF regression

- **Bias–variance trade-off:** small λ = low bias, high variance; large λ = high bias, low variance.
- Optimal λ selected by cross-validation.

Model Selection: The Bias–Variance Trade-off



Training error decreases monotonically with polynomial degree.

Generalisation error is U-shaped.

- **Underfitting (high bias):** Model too simple, misses important trends.
- **Overfitting (high variance):** Model too complex, fits noise, poor generalisation.
- **Generalisation error:**

$$\epsilon_{\text{gen}} = \mathbb{E}[(y - \hat{f}(\mathbf{x}))^2]$$

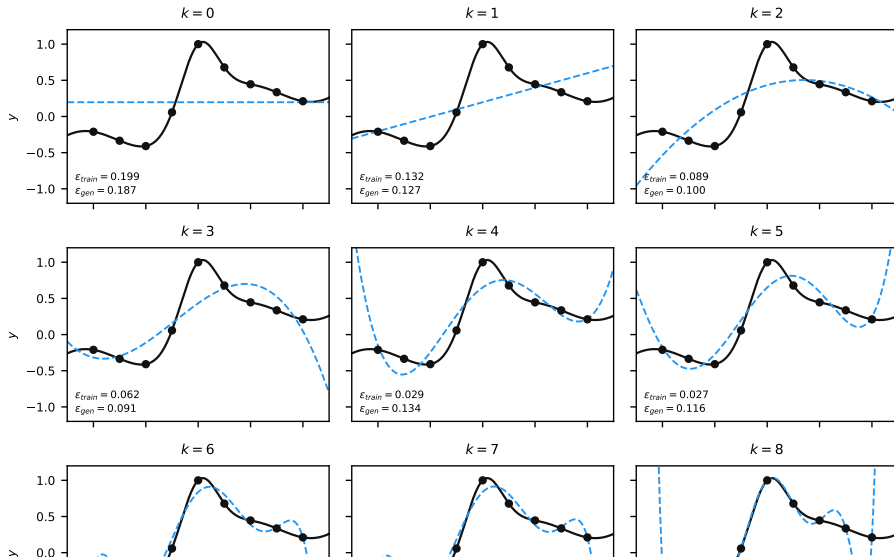
- True generalisation error is *unknowable* exactly — we use empirical estimates.

Example 17.1

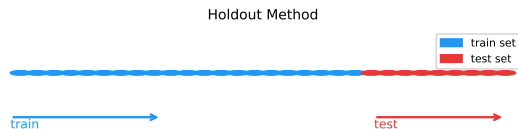
$f(x) = x/10 + \sin(x)/4 + e^{-x^2}$
9 points in $[-4, 4]$; evaluated on $[-5, 5]$.
Optimal degree ≈ 3 .

Polynomial Degree: Training vs Generalisation

Polynomial Surrogate: Degree vs Error



Model Selection: Holdout Method



Split dataset into train (blue) and test (red).

Procedure

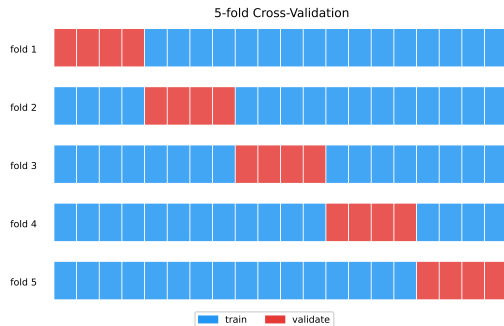
- 1 Split m samples: S_{train} ($\sim 70\text{--}80\%$) and S_{test} .
- 2 Fit model on S_{train} .
- 3 Evaluate on S_{test} to estimate ϵ_{gen} .

```
1 import numpy as np
2
3 def holdout_split(m, frac=0.7, seed=0):
4     """Returns (train_idx, test_idx)."""
5     rng = np.random.default_rng(seed)
6     perm = rng.permutation(m)
7     n_train = int(frac * m)
8     return perm[:n_train], perm[n_train:]
9
10 def holdout_estimate(X, y, bases, seed=0):
11     tr, te = holdout_split(len(X), seed=seed)
12     model = regression(X[tr], y[tr], bases)
13     preds = np.array([model(x) for x in X[te]])
14     return np.mean((y[te] - preds)**2)
```

Listing 4: Holdout method (Algorithm 17.7)

- Simple and fast.
- **Drawback:** result depends on which samples fall in test set.
- Solution: average over multiple random splits (cross-validation).

k-Fold Cross-Validation



5-fold CV: each fold takes a turn as the validation set.

Algorithm (Alg. 17.10)

- 1 Partition m samples into k folds.
- 2 For $i = 1, \dots, k$:
 - Train on all folds except fold i .
 - Validate on fold i .
- 3 Estimate $\epsilon_{\text{gen}} = \frac{1}{k} \sum_{i=1}^k \epsilon_{\text{test}}^{(i)}$.

- Uses *all* data for both training and validation.
- **Leave-one-out CV** ($k = m$): deterministic, trains m models.
- Common choices: $k = 5$ or $k = 10$.

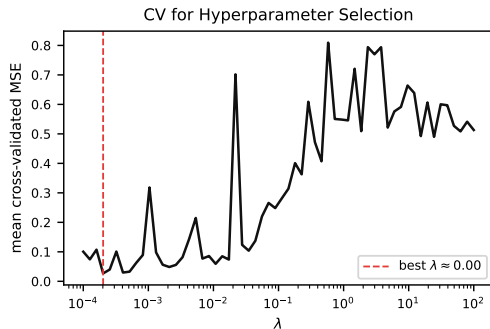
Complexity

Trains k models, each on $(k - 1)/k$ fraction of data.
Total cost: $O(k \cdot \text{fit cost})$.

Cross-Validation in Python

```
1 import numpy as np
2 from dataclasses import dataclass
3 from typing import List
4
5 @dataclass
6 class TrainTest:
7     train: np.ndarray
8     test: np.ndarray
9
10 def k_fold_cv_sets(m: int, k: int) -> List[TrainTest]:
11     """Generate k train/test index splits."""
12     perm = np.random.permutation(m)
13     fold_size = m // k
14     sets = []
15     for i in range(k):
16         test_idx = perm[i*fold_size:(i+1)*fold_size]
17         train_idx = np.concatenate([perm[:i*fold_size],
18                                     perm[(i+1)*fold_size:]])
19         sets.append(TrainTest(train_idx, test_idx))
20     return sets
21
22 def cv_estimate(X, y, sets, bases, lam=0.0):
23     """Mean CV error across all folds."""
24     errors = []
25     for s in sets:
26         model = regression(X[s.train], y[s.train], bases, lam)
27         preds = np.array([model(x) for x in X[s.test]])
```

Cross-Validation for Hyperparameter Selection (Example 17.2)

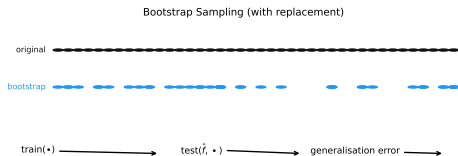


Mean cross-validated MSE vs. λ for 10 noisy samples of $f(x) = \sin(2x) \cos(10x) + \epsilon/10$. Minimum near $\lambda \approx 0.2$.

```
1 import numpy as np
2
3 np.random.seed(0)
4 m = 10
5 X = np.random.rand(m)
6 y = np.sin(2*X)*np.cos(10*X) + np.random.randn(m)/10
7
8 # Gaussian RBF basis
9 c = 5.0
10 bases = [lambda x, xi=xi: np.exp(-c*(x-xi)**2)
11           for xi in X]
12
13 # 3-fold CV over lambda grid
14 sets = k_fold_cv_sets(m, k=3)
15 lambdas = np.logspace(-4, 2, 60)
16 cv_errs = [cv_estimate(X, y, sets, bases, lam)
17            for lam in lambdas]
18
19 best_lam = lambdas[np.argmin(cv_errs)]
20 print(f"Best lambda = {best_lam:.4f}")
```

- Sweep λ over logarithmic grid.
- Select model minimising CV error.
- Result: $\lambda_{\text{best}} \approx 0.2$.

The Bootstrap Method



Bootstrap: sample m indices *with replacement*, train on bootstrap sample, test on full dataset.

Bootstrap Error Estimate (Eq. 17.31–17.32)

If b bootstrap samples are used:

$$\epsilon_{\text{boot}} = \frac{1}{b} \sum_{i=1}^b \epsilon_{\text{test}}^{(i)} = \frac{1}{m} \sum_{j=1}^m \frac{1}{b} \sum_{i=1}^b (y^{(j)} - \hat{f}^{(i)}(\mathbf{x}^{(j)}))^2$$

Leave-One-Out Bootstrap (Eq. 17.33)

$$\epsilon_{\text{loo-boot}} = \frac{1}{m} \sum_{j=1}^m \frac{1}{c_{-j}} \sum_{\substack{i=1 \\ j \notin S^{(i)}}}^b (y^{(j)} - \hat{f}^{(i)}(\mathbf{x}^{(j)}))^2$$

where c_{-j} = number of bootstrap samples not containing j .

Bootstrap in Python

```
1 import numpy as np
2
3 def bootstrap_sets(m: int, b: int):
4     """Generate b bootstrap (train, test=all) pairs."""
5     return [(np.random.choice(m, m, replace=True),
6             np.arange(m)) for _ in range(b)]
7
8 def bootstrap_estimate(X, y, b_sets, bases, lam=0.0):
9     """Bootstrap generalisation error estimate."""
10    errors = []
11    for train, test in b_sets:
12        model = regression(X[train], y[train], bases, lam)
13        preds = np.array([model(x) for x in X[test]])
14        errors.append(np.mean((y[test] - preds)**2))
15    return np.mean(errors)
16
17 def loo_bootstrap_estimate(X, y, b_sets, bases, lam=0.0):
18     """Leave-one-out bootstrap estimate."""
19    m = len(X)
20    models = [regression(X[tr], y[tr], bases, lam)
21              for tr, _ in b_sets]
22    eps = 0.0
23    for j in range(m):
24        c_j = sum(1 for i, (tr, _) in enumerate(b_sets)
25                  if j not in tr)
26        if c_j == 0:
27            continue
```

Information-Theoretic Model Selection: AIC and BIC

Akaike Information Criterion (AIC)

$$\text{AIC} = 2q - 2 \ln \hat{\mathcal{L}}$$

- q : number of model parameters.
- $\hat{\mathcal{L}}$: maximum likelihood.
- For Gaussian noise: $\text{AIC} = n \ln(\text{SSE}/n) + 2q$.
- Penalises complexity mildly.

Bayesian Information Criterion (BIC)

$$\text{BIC} = q \ln n - 2 \ln \hat{\mathcal{L}}$$

- Penalises complexity more heavily than AIC (factor $\ln n$ vs 2).
- Tends to select *sparser* models.

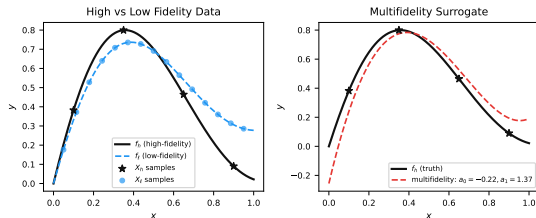
Multifidelity Surrogate Models

Motivation

Often multiple evaluation sources are available:

- f_h : **high-fidelity** (expensive, accurate) – e.g. full CFD.
- f_ℓ : **low-fidelity** (cheap, biased) – e.g. reduced-order model.

We have many evaluations X_ℓ from f_ℓ but few from X_h .



Left: high vs low fidelity functions. Right: linear correction multifidelity surrogate.

Linear Multifidelity Model (Eq. 17.36)

$$f_h(\mathbf{x}) \approx a_0 + a_1 \hat{f}_\ell(\mathbf{x})$$

- $\hat{f}_\ell$: surrogate fitted to low-fidelity data X_ℓ .
- a_0, a_1 : correction parameters fitted using X_h .
- $a_1 \approx$ scaling; $a_0 \approx$ bias correction.

Multifidelity Model: Construction and Python

```
1 import numpy as np
2
3 def multifidelity_surrogate(X_lo, y_lo,
4                             X_hi, y_hi,
5                             lo_bases):
6     """
7     Build multifidelity surrogate.
8     Step 1: fit low-fidelity model
9     Step 2: use low-fi predictions at X_hi
10             to fit linear correction a0, a1
11     """
12     # Step 1: low-fidelity surrogate
13     f_lo = regression(X_lo, y_lo, lo_bases)
14
15     # Step 2: linear correction
16     lo_at_hi = np.array([f_lo(x) for x in X_hi])
17     A = np.column_stack([np.ones(len(X_hi)),
18                          lo_at_hi])
19     params, _, _, _ = np.linalg.lstsq(A, y_hi,
20                                       rcond=None)
21     a0, a1 = params
22
23     # Combined multifidelity model
24     def mf_model(x):
25         return a0 + a1 * f_lo(x)
26     return mf_model, (a0, a1)
```

Listing 7: Linear multifidelity surrogate

Procedure Summary

- 1 Fit $\hat{f}_\ell$ on large X_ℓ dataset.
 - 2 Evaluate $\hat{f}_\ell(\mathbf{x}^{(i)})$ at all high-fidelity points $\mathbf{x}^{(i)} \in X_h$.
 - 3 Regress $y_h^{(i)}$ against $\hat{f}_\ell(\mathbf{x}^{(i)})$ to find a_0, a_1 .
 - 4 Final model: $\hat{f}(\mathbf{x}) = a_0 + a_1 \hat{f}_\ell(\mathbf{x})$.
- Extension: non-hierarchical datasets, Bayesian multifidelity.
 - Application: aircraft collision avoidance (Exercise 17.5):

$$a_0 \approx 0, \quad a_1 \approx 0.01$$

Multifidelity: Cases and Considerations

Case 1: Hierarchical (most common)

f_h and f_ℓ measure the *same* quantity at different accuracy levels.

- Low fidelity is a fast approximation (coarser mesh, fewer Monte Carlo samples).
- Linear correction typically adequate.

Case 2: Non-comparable outputs

f_ℓ and f_h measure *related but different* quantities.

- Example: f_ℓ = sweetness/salinity proxy; f_h = customer satisfaction.
- Build a *mapping* between outputs using shared design samples.

Case 3: Non-hierarchical datasets

Multiple sources of unknown or variable fidelity.

- Build separate surrogates for each source.
- Combine predictions (weighted averaging, Bayesian model averaging).

Chapter 17 Summary

Key Takeaways

- 1 **Surrogate models** approximate expensive objective functions using cheap, smooth models fitted by regression.
- 2 **Regression** minimises $\|\mathbf{y} - \mathbf{B}\boldsymbol{\theta}\|_2^2$; analytic solution $\boldsymbol{\theta} = \mathbf{B}^+\mathbf{y}$.
- 3 **Basis functions** extend linear models to nonlinear surrogates: polynomial, sinusoidal (Fourier), radial (RBF).
- 4 **Noisy objectives**: L_2 regularisation ($\lambda > 0$) prevents overfitting; optimal λ via cross-validation.
- 5 **Model selection** balances bias (underfitting) and variance (overfitting) using holdout, k -fold CV, bootstrap, AIC, or BIC.
- 6 **Multifidelity** surrogates exploit cheap low-fidelity data to boost accuracy with few expensive high-fidelity evaluations.

Practical Guidelines

- Start with linear or low-degree polynomial.
- Switch to RBF for complex nonlinearity.

Connections to Other Chapters

- Chapter 16: sampling plans to build X .
- Chapter 18: Bayesian optimisation uses GP_{30/34}

Exercises (Chapter 17) I

Exercise 17.1 – Normal Equation

Derive an expression satisfied by the optimum of $\min_{\theta} \|\mathbf{y} - \mathbf{X}\theta\|_2^2$ by setting the gradient to zero.

Solution:

$$\nabla_{\theta}(\mathbf{y} - \mathbf{X}\theta)^{\top}(\mathbf{y} - \mathbf{X}\theta) = -2\mathbf{X}^{\top}(\mathbf{y} - \mathbf{X}\theta) = \mathbf{0} \implies \mathbf{X}^{\top}\mathbf{X}\theta = \mathbf{X}^{\top}\mathbf{y}$$

Exercise 17.2 – When to Use Complex Models

When would we use a more descriptive model (polynomial) versus simpler (linear)?

Solution: Use more complex models when more data are available. With few samples, simpler models with fewer degrees of freedom prevent overfitting.

Exercise 17.3 – Iterative Solvers for Large Regression

Why might we use iterative methods instead of the analytic solution?

Solution: The design matrix $\mathbf{B} \in \mathbb{R}^{m \times q}$ can be extremely large. The normal equations require $O(q^3)$ factorisation; iterative methods (stochastic gradient descent) have $O(q)$ memory and are the only viable option at scale.

Exercises (Chapter 17) III

Exercise 17.4 – Leave-One-Out CV

Evaluate f at $x = 1, 2, 3, 4$ giving $y = 0, 5, 4, 6$. Compute the LOOCV MSE for polynomial degrees $k = 0, \dots, 4$.

Solution: LOOCV with $k = 4$ folds. Minimum MSE achieved at $k = 1$ (linear model), giving $\theta \approx [-0.5, 1.7]$, i.e. $\hat{f}(x) = -0.5 + 1.7x$.

Exercise 17.5 – Aircraft Collision Avoidance

f_ℓ : 1 000 flight hours (biased scenarios, 100% safety-critical). f_h : 1 000 000 flight hours (1% safety-critical). Expected a_0 and a_1 in $f_h \approx a_0 + a_1 f_\ell$?

Solution: f_ℓ overestimates collision rate by $\sim 100\times$ (all scenarios critical). Therefore $a_1 \approx 0.01$ to scale down; $a_0 \approx 0$ (no additive bias needed).

References

- M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed. MIT Press, 2026. Chapter 17: Surrogate Models (pp. 365–388).
- K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- P. Jiang, Q. Zhou, and X. Shao, *Surrogate Model-Based Engineering Design and Optimization*. Springer, 2020.
- A. Forrester, A. Sobester, and A. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*. Wiley, 2008.
- M. Stone, “Cross-Validatory Choice and Assessment of Statistical Predictions,” *J. Royal Statistical Society*, vol. 36, no. 2, pp. 111–147, 1974.
- B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, pp. 1–26, 1979.